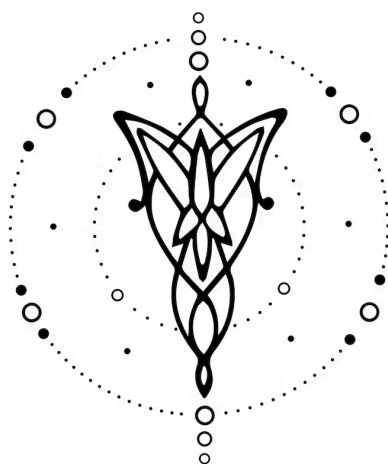




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Valentina	Fernández Flores	64142	vfernandezflores@itba.edu.ar
Julia	Priolo	63743	jupriolo@itba.edu.ar
Olivia	Rodrigué	64180	orodrigue@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: **TPE-TLA**

3. Dominio

En la actualidad, la visualización de datos en entornos web requiere la combinación de tecnologías como **HTML, CSS y JavaScript**, junto con librerías de terceros (por ejemplo, **Chart.js** o **D3.js**). Este proceso, si bien es potente, suele resultar complejo para usuarios sin formación técnica o que simplemente buscan generar gráficos de manera rápida y sencilla.

El propósito del presente trabajo es diseñar un **Lenguaje Específico de Dominio (DSL)** que permita **describir y procesar datos provenientes de archivos CSV para generar gráficos web de forma declarativa**. El lenguaje no solo facilitará la creación de visualizaciones, sino que también ofrecerá mecanismos para **transformar y preparar los datos antes de graficarlos**, mejorando así su versatilidad y expresividad.

El DSL permitirá:

- Parsear tanto el **programa fuente** como el **archivo CSV** de entrada.
- Aplicar **operaciones de filtrado, proyección y agregación** sobre los datos antes de su visualización.
- Reutilizar conjuntos de datos procesados a través de **fuentes (source) definidas por el usuario**, que podrán combinarse o transformarse entre sí.
- Generar automáticamente código HTML y JavaScript válidos, reduciendo el esfuerzo manual y la posibilidad de errores.

El dominio, entonces, se centra en la **simplificación integral del proceso de generación de gráficos web**, abarcando tanto la manipulación de datos como su representación visual. El lenguaje apunta a ser **extensible**, permitiendo agregar en el futuro nuevos tipos de gráficos, operaciones o librerías de renderizado sin alterar la base del compilador.

4. Construcciones

El lenguaje propone una sintaxis declarativa que combina tres niveles principales:

1. Definición de fuentes de datos (sources)

- Permite cargar y procesar archivos CSV mediante operaciones básicas de:
 - **Filtrado:** selección de filas según predicados basados en los nombres de columnas.

- **Proyección:** selección de columnas específicas, útil para gráficos con dimensiones limitadas o múltiples series.
- **Agregación:** operaciones sobre columnas (ej. COUNT, MIN, MAX, AVERAGE), sin necesidad de GROUP BY explícitos.

Ejemplo:

```
source ventas_filtradas = from "ventas.csv"
filter "Región" == "Norte"
project ["Mes", "Monto"];
```

2. Creación de gráficos (chart)

Cada gráfico puede consumir directamente un archivo CSV o una source previamente definida.

Se soportan los tipos: pie, donut, bar, scatter y **line** (nuevo tipo agregado).

Se permiten operaciones vectoriales y escalares sobre las columnas, como:

```
y = "Peso" / ("Altura" * "Altura") as "BMI"
```

o conversiones simples:

```
y = "Peso" / 1000 as "Peso [kg]"
```

3. Parámetros visuales y estructurales

- title: título del gráfico.
- colors: lista de colores (sin comillas) definidos con formato hexadecimal (#RRGGBB).
- legend: posición (top, bottom, left, right) o off.
- orientation: vertical o horizontal.
- id: identificador único.
- Parámetros específicos como hole para los gráficos tipo donut (ejemplo: hole = 0.2).

5. Casos de Prueba

5. Casos de Prueba

Casos de Aceptación

1. Gráfico de barras a partir de ventas.csv con etiquetas y valores.
2. Gráfico de torta con tres categorías.

3. Gráfico de dispersión con pares ordenados.
4. Gráfico de anillo con colores personalizados y parámetro hole.
5. Gráfico de líneas mostrando la evolución de ventas a lo largo del tiempo.
6. Programa que genera múltiples gráficos (ej. bar y pie) en un mismo HTML.
7. Gráfico de barras con título definido por el usuario.
8. Gráfico con leyenda activada y posición inferior.
9. Gráfico que utiliza una source filtrada.
10. Gráfico con operaciones escalares sobre columnas (por ejemplo, conversión de unidades).
11. Generación de un HTML válido, con datos procesados y optimizados en la salida.

Casos de Rechazo

1. Errores de sintaxis.
2. Gráfico sin especificar tipo.
3. Archivo CSV inexistente.
4. Tipo de gráfico no soportado.
5. Identificadores duplicados.
6. Referencias a columnas inexistentes.
7. Uso incorrecto de operaciones sobre columnas.

Ejemplo 1 – Gráfico de barras simple

chart "Ventas 2024" type bar:

```
from "ventas.csv",
x = "Mes",
y = "Monto",
orientation = vertical,
colors = [#4CAF50, #2196F3],
id = "grafico_ventas";
```

Ejemplo 2 – Gráfico de dispersión

chart "Altura vs Peso" type scatter:

```
from "datos_personas.csv",
x = "Peso",
y = "Altura",
x.range = [50, 100],
y.range = [150, 200],
color = #FF0000,
id = "grafico_altura_peso";
```

Ejemplo 3 – Gráfico de torta

chart "Tartas 2025" type pie:

```
from "tartas_2025.csv",
x = "Tipo de tartas",
y = "Cantidad",
legend = bottom,
id = "tipo_tartas";
```

Ejemplo 4 – Gráfico de anillo

chart "Distribución de gastos mensuales" type donut:

```
from "gastomensuales.csv",
x = "Gastos",
y = "Porcentaje",
hole = 0.2,
id = "grafico_donut";
```

Ejemplo 5 – Definición y reutilización de fuentes

source datos_filtrados = from "personas.csv"

```
filter "Edad" > 18
```

```
project ["Edad", "Altura", "Peso"];
```

chart "BMI Promedio" type line:

```
from datos_filtrados,
x = "Edad",
y = "Peso"/("Altura"*"Altura") as "BMI",
colors = [#8E44AD],
id = "grafico_bmi";
```

Ejemplo 6 – Agregación (promedio de ventas por mes)

chart "Promedio de ventas por mes" type line:

```
from "ventas.csv",
x = "Mes",
y = average("Monto"),
colors = [#2980B9],
id = "grafico_promedio";
```

Ejemplo 7 – Múltiples series en un mismo gráfico

source ventas_norte = from "ventas_norte.csv" project ["Mes", "Monto"];

source ventas_sur = from "ventas_sur.csv" project ["Mes", "Monto"];

chart "Comparación de ventas por región" type line:

```
from [ventas_norte, ventas_sur],
x = "Mes",
y = "Monto",
colors = [#E74C3C, #2ECC71],
id = "grafico_series";
```

Ejemplo 8 – Reutilización de sources con composición

```
source base = from "ventas.csv"  
  filter "Año" == 2024;
```

```
source mayores_a_1000 = from base  
  filter "Monto" > 1000  
  project ["Mes", "Monto"];
```

```
chart "Ventas mayores a 1000" type bar:  
  from mayores_a_1000,  
  x = "Mes",  
  y = "Monto",  
  colors = [#9B59B6],  
  id = "grafico_filtrado";
```